



DOCUMENTATION TECHNIQUE

Déploiement et exploitation

Mise en ligne, sauvegarde, restauration et supervision

Ce document couvre les environnements **staging** et **production**, la procédure de sauvegarde/restoration et la stratégie de retour arrière.

Sommaire

1. Architecture cible

2. Variables d'environnement de production

3. Déploiement par Docker (recommandé)

Déploiement sans Docker

4. Schéma de base de données

5. Reverse proxy (exemple Nginx)

6. Environnement de staging

7. Sauvegarde

8. Restauration

9. Rollback

10. Supervision

11. E-mails et newsletter : délivrabilité

12. Vente directe : Stripe et PayPal

12 bis. Google Sheets (facultatif)

13. Checklist de mise en ligne

1. Architecture cible

L'application est un **serveur Node.js unique** qui sert à la fois :

- le site public (rendu statique + serveur) ;
- l'administration CMS (`/admin`) ;
- l'API du formulaire (`/api/contact`) et l'API du CMS (`/api/cms`).

Elle a besoin de :

Composant	Rôle	Remarque
Node.js ≥ 20.9	Exécution	<code>npm start</code> (port 4313 par défaut)
PostgreSQL 16	Contenus, utilisateurs, demandes de contact	Sauvegarde quotidienne
Volume disque persistant	Médias (<code>public/media</code>) et fichiers vendus (<code>private</code>)	Indispensable : un hébergement à système de fichiers éphémère (Vercel, Netlify) perdrait les images téléversées → utiliser un stockage objet ou un VPS
SMTP	E-mails du formulaire	Fournisseur au choix
Reverse proxy (Nginx / Apache / Caddy)	HTTPS, compression, cache statique	Certificat Let's Encrypt

Hébergement recommandé : **VPS Linux** (2 vCPU / 4 Go) avec Docker, ou toute plateforme supportant un serveur Node persistant et un volume.

2. Variables d'environnement de production

Modèle complet et commenté : `.env.production.example` (versionné). Sur le serveur :

```
cp .env.production.example .env.production
chmod 600 .env.production      # jamais versionné (.gitignore) ni copié dans l'image (.dockerignore)
```

Variable	Rôle
<code>NEXT_PUBLIC_SITE_URL</code>	URL publique, sans barre finale. Inscrite dans le build : reconstruire l'image après un changement de domaine
<code>POSTGRES_USER</code> , <code>POSTGRES_PASSWORD</code> , <code>POSTGRES_DB</code>	Base PostgreSQL ; l'adresse de connexion de l'application en est déduite. Mot de passe en lettres et chiffres
<code>PAYLOAD_SECRET</code>	48 octets aléatoires, différent entre staging et production
<code>EMAIL_ENABLED</code> , <code>SMTP_*</code> , <code>EMAIL_FROM</code> , <code>EMAIL_TO</code>	Envoi des e-mails du formulaire ; laisser <code>EMAIL_ENABLED=false</code> tant qu'un envoi réel n'a pas été testé
<code>CONTACT_RATE_LIMIT</code> , <code>CONTACT_RATE_WINDOW_MINUTES</code>	Anti-abus du formulaire (5 envois / 15 min)
<code>CONTACT_RETENTION_MONTHS</code>	Suppression automatique des demandes de contact inchangées depuis N mois (24 par défaut ; <code>0</code> = jamais)
<code>NEXT_PUBLIC_ANALYTICS_*</code>	Mesure d'audience facultative (inscrite dans le build) : <code>umami</code> / <code>plausible</code> (+ <code>SCRIPT_URL</code>), ou <code>google</code> + <code>SITE_ID=G-...</code> (bannière de consentement, domaines Google ajoutés à la CSP)
<code>SEED_ADMIN_EMAIL</code> , <code>SEED_ADMIN_PASSWORD</code>	Premier administrateur, créé par <code>npm run seed</code> ; à retirer du fichier ensuite

Points de vigilance :

- L'application **refuse de démarrer** si `PAYLOAD_SECRET` est absent, trop court ou laissé à sa valeur d'exemple (le build, lui, n'en a pas besoin).
- Aucun secret dans le dépôt ni dans l'image : ils sont injectés au démarrage.

3. Déploiement par Docker (recommandé)

Fichiers fournis à la racine du dépôt :

Fichier	Contenu
<code>Dockerfile</code>	Image Node 22 en 3 étapes ; utilisateur non root ; au démarrage : migrations de base (<code>npm run migrate</code>) puis serveur ; healthcheck
<code>docker-compose.prod.yml</code>	Application + PostgreSQL 16, volumes <code>media</code> , <code>private</code> et <code>db</code> , application exposée sur <code>127.0.0.1</code> uniquement (derrière le proxy)
<code>.dockerignore</code>	Exclut secrets, médias, dépendances et résultats de tests du contexte de build

Pour alléger les commandes, définir une fois l'alias :

```
alias dc='docker compose --env-file .env.production -f docker-compose.prod.yml'
```

Première mise en ligne

```
git clone git@github.com:carine2107/RK-Portfolio.git /opt/romial && cd /opt/romial
cp .env.production.example .env.production && chmod 600 .env.production # puis le remplir
dc up -d --build # construit l'image, crée les tables, démarre
dc exec app npm run seed # une seule fois : contenus de départ + premier administrateur
```

Photos et couverture du livre (facultatif, une fois) : copier les fichiers sur le serveur puis `dc exec app npm run import:assets -- "<dossier photos>" "<couverture>"` (voir `INSTALLATION.md`). Sinon, les téléverser depuis l'administration.

Mise à jour

```
cd /opt/romial
ops/backup.sh # sauvegarde avant toute mise à jour (section 7)
git pull
dc up -d --build # les migrations en attente s'appliquent au démarrage
```

Le volume `media` est **obligatoire** : sans lui, chaque redéploiement effacerait les images téléversées depuis le CMS.

L'image est construite **sans accès à la base** : les pages ne sont donc pas pré-générées pendant le build. Chacune est rendue depuis le CMS à sa première visite, puis servie depuis le cache (renouvelé toutes les 5 minutes). Le site n'affiche jamais le contenu de démarrage intégré au code.

Déploiement sans Docker

Serveur Node ≥ 20.9 et PostgreSQL 16 ; variables de la section 2 dans `.env` (avec `NODE_ENV=production` et `DATABASE_URI`).

```
git pull
npm ci
npm run migrate # applique les migrations en attente
npm run build # la base étant joignable, les pages sont pré-générées
pm2 restart romial-site # ou systemd
```

4. Schéma de base de données

- En **développement**, Payload synchronise le schéma automatiquement.
- En **production**, cette synchronisation est désactivée : le schéma évolue uniquement par **migrations versionnées** (`src/migrations/`). La première, `initial`, crée toutes les tables d'une base vierge.
- Le conteneur applique les migrations en attente à chaque démarrage (`npm run migrate`). Sans Docker, lancer `npm run migrate` avant le build.

Après toute évolution du modèle de contenu (nouveau champ, nouvelle collection) :

```
npm run migrate:create <nom> # sur le poste de développement ; versionner le fichier créé
```

Ordre d'un déploiement avec migration : **sauvegarde** → **mise à jour du code** → **migration** → **redémarrage** (automatique avec Docker).

Ne jamais lancer `npm run migrate` sur la base de développement locale : elle est gérée par la synchronisation automatique, et Payload demanderait une confirmation pouvant entraîner une perte de données.

5. Reverse proxy (exemple Nginx)

```
server {
    listen 443 ssl http2;
    server_name romialkenmogne.com;

    ssl_certificate      /etc/letsencrypt/live/romialkenmogne.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/romialkenmogne.com/privkey.pem;

    client_max_body_size 12M; # téléversements CMS (limite applicative : 10 Mo)

    location / {
        proxy_pass http://127.0.0.1:4313;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /_next/static/ {
        proxy_pass http://127.0.0.1:4313;
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
}

server {
    listen 80;
    server_name romialkenmogne.com;
    return 301 https://$host$request_uri;
}
```

`X-Forwarded-For` est nécessaire : la limitation de débit du formulaire s'appuie sur cet en-tête.

6. Environnement de staging

Identique à la production, avec :

- un sous-domaine (`staging.romialkenmogne.com`) ;
- **sa propre base** et son propre `PAYLOAD_SECRET` ;
- un blocage de l'indexation au niveau du proxy :

```
add_header X-Robots-Tag "noindex, nofollow" always;
```

Le staging sert à la recette du commanditaire avant chaque mise en production.

7. Sauvegarde

Trois éléments à sauvegarder : **la base, les médias** (volume `media`) et **les fichiers vendus** (volume `private` : e-books, ressources, pièces jointes des leçons). Le script fourni `ops/backup.sh` sauvegarde les trois, sans supposer le nom des volumes sur le disque (il les lit à travers le conteneur de l'application) :

```
chmod +x /opt/romial/ops/*.sh
/opt/romial/ops/backup.sh      # test manuel
crontab -e                    # puis ajouter la ligne suivante (root)
0 3 * * * /opt/romial/ops/backup.sh >> /var/log/romial-backup.log 2>&1
```

Chaque sauvegarde produit, dans `/var/backups/romial` (variable `BACKUP_DIR`) :

Fichier	Contenu
<code>db-AAAA-MM-JJ_HHMM.dump</code>	Base PostgreSQL (<code>pg_dump</code>), format personnalisé, sans propriétaire
<code>files-AAAA-MM-JJ_HHMM.tar.gz</code>	<code>public/media</code> et <code>private</code> (propriétaire et droits conservés)
<code>backup-AAAA-MM-JJ_HHMM.sha256</code>	Empreintes SHA-256, vérifiées avant toute restauration

Une sauvegarde interrompue ne laisse que des fichiers `.partial`, jamais une sauvegarde apparemment complète. Au-delà de `RETENTION_DAYS` (30 jours par défaut), les anciennes sauvegardes sont supprimées.

Élément	Fréquence	Rétention	Vérification
Base PostgreSQL	quotidienne	30 jours	restauration testée chaque trimestre
Médias et fichiers vendus	quotidienne	30 jours	idem
Dépôt Git	à chaque commit	illimitée	miroir distant

Copier les sauvegardes hors du serveur chaque jour (stockage objet, autre machine ou poste du commanditaire) : une sauvegarde restée sur le serveur disparaît avec lui. Elles contiennent des données personnelles (demandes de contact, abonnés, commandes) : stockage chiffré et accès restreint.

8. Restauration

```
/opt/romial/ops/restore.sh      # liste les sauvegardes disponibles
/opt/romial/ops/restore.sh 2026-09-15_0300 # restaure, après avoir tapé RESTAURER
```

Le script vérifie les empreintes (une sauvegarde altérée est refusée), arrête le site, **remplace** la base (`pg_restore --clean`, en une seule transaction : en cas d'erreur, la base reste dans son état précédent), remplace les médias et les fichiers vendus, redémarre le site puis attend que `/api/health` réponde.

- Restaurer avec la **version du site** en service au moment de la sauvegarde (tag Git, section 9), surtout après une migration de schéma.

- Sur un **nouveau serveur** : installer le site (section 3, sans `npm run seed`), copier les trois fichiers dans `/var/backups/romial`, puis lancer `ops/restore.sh`.

Contrôles après restauration : page d'accueil dans les trois langues, connexion à `/admin`, présence des images, téléchargement d'un produit acheté, envoi d'une demande de test. Test de restauration réel : voir `RAPPORT_TESTS.md`.

9. Rollback

Situation	Action
Régression applicative, schéma inchangé	<code>git checkout <tag-précédent></code> puis <code>dc up -d --build</code> (sans Docker : <code>npm ci && npm run build</code> et redémarrage)
Régression après migration de schéma	Restaurer la sauvegarde de base prise avant la migration , puis redéployer la version précédente
Contenu supprimé par erreur	Payload conserve les versions : ouvrir l'entrée → onglet <i>Versions</i> → <i>Restore</i>
Incident majeur	Restauration complète (section 8) puis analyse hors production

Marquer chaque mise en production par un tag Git (`git tag -a v1.0.0`) : le rollback consiste alors simplement à redéployer le tag précédent.

10. Supervision

Élément	Mise en place
Point de santé	<code>GET /api/health</code> → <code>200 {"status":"ok","database":"up"}</code> , ou <code>503</code> si la base est injoignable. Aucune information sensible. Utilisé par le healthcheck Docker
Surveillance de disponibilité	Sonde externe sur <code>https://<domaine>/api/health</code> toutes les 1 à 5 min avec alerte e-mail : UptimeRobot ou Better Stack (offres gratuites), ou Uptime Kuma auto-hébergé sur une autre machine
Journaux	<code>dc logs -f app</code> ; le formulaire journalise le type de demande et le résultat, jamais le contenu ni l'adresse e-mail. Rotation : <code>"log-opts": {"max-size": "10m", "max-file": "5"}</code> dans <code>/etc/docker/daemon.json</code>
Erreurs	Les erreurs serveur apparaissent dans les journaux avec leur identifiant (<code>digest</code>) ; un service de suivi (Sentry, GlitchTip auto-hébergé) peut être branché via <code>src/instrumentation.ts</code>
Tâche quotidienne	Suppression des demandes de contact expirées (<code>CONTACT_RETENTION_MONTHS</code>), lancée par le serveur ; manuellement : <code>dc exec app npm run purge:contacts</code>

11. E-mails et newsletter : délivrabilité

Le formulaire de contact et la newsletter partagent le compte SMTP (`SMTP_*`). Pour que les e-mails arrivent en boîte de réception :

- utiliser un fournisseur transactionnel (Brevo, Postmark, Mailgun...) ; vérifier sa **limite d'envoi quotidienne** au regard du nombre d'abonnés ;
- publier les enregistrements DNS **SPF**, **DKIM** (fournis par le fournisseur) et **DMARC** (`v=DMARC1; p=none; rua=mailto:...` pour commencer) ;
- `EMAIL_FROM` sur le domaine du site (`no-reply@romialkenmogne.com`), jamais une adresse Gmail ;
- tester l'envoi avec mail-tester.com (score visé ≥ 9/10).

Les e-mails d'article portent les en-têtes `List-Unsubscribe` et `List-Unsubscribe-Post` (désinscription en un clic, exigée par Gmail et Yahoo pour les envois en nombre). Les liens de confirmation et de désinscription sont signés avec `PAYLOAD_SECRET` : **changer ce secret invalide les liens déjà envoyés.**

12. Vente directe : Stripe et PayPal

Variables (`.env.production`) : `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`, `PAYPAL_MODE` (`live` en production). Un prestataire sans ses clés n'est pas proposé ; sans aucun prestataire, la boutique reste fermée même cochée dans le CMS.

Stripe (comptes au nom du commanditaire) :

1. Dashboard → Developers → **API keys** : clé secrète (`sk_live_...`, ou `sk_test_...` sur le staging) — ou une clé restreinte limitée à *Checkout Sessions* : *write*.
2. Developers → **Webhooks** → endpoint `https://<domaine>/api/shop/stripe/webhook`, événements `checkout.session.completed`, `checkout.session.async_payment_succeeded` et `checkout.session.expired` ; copier le **signing secret** (`whsec_...`).
3. Settings → **Emails** : activer l'envoi automatique des reçus.
4. Test sur le staging avec la carte `4242 4242 4242 4242` (mode test), puis vérifier que la commande passe « Payée » et que les e-mails arrivent.

PayPal : `developer.paypal.com` → **Apps & Credentials** → application REST (Client ID / Secret, en *Sandbox* puis *Live*). Aucun webhook requis : le paiement est capturé au retour de l'acheteur sur le site.

Sécurité : aucune donnée de carte ne transite par le site (pages de paiement hébergées) ; les montants sont recalculés côté serveur à partir du CMS ; une commande ne passe « Payée » que sur un webhook dont la signature est vérifiée ou une capture PayPal réussie, une seule fois.

Produits numériques et espace membres : les fichiers vendus (e-books, ressources, pièces jointes des leçons) sont stockés dans `/app/private/files`, monté sur le volume Docker `private` — à inclure dans les sauvegardes au même titre que `media`. Ils ne sont jamais servis publiquement. La connexion des acheteurs se fait par lien e-mail : le SMTP doit donc fonctionner, et **changer** `PAYLOAD_SECRET` **déconnecte tous les membres** et invalide les liens envoyés. Seules les commandes contenant un livre imprimé demandent une adresse de livraison.

12 bis. Google Sheets (facultatif)

Copie automatique des **demandes de contact** et des **abonnés newsletter confirmés** dans un tableur Google. Fonctionne avec un **compte Google gratuit**, sans carte bancaire. Sans les trois variables ci-dessous, rien n'est envoyé à Google.

1. Le tableur — sur `https://sheets.google.com`, créer un tableur vide (par exemple « RK — Contacts »). Son **identifiant** est la partie de l'adresse entre `/d/` et `/edit` : `https://docs.google.com/spreadsheets/d/<IDENTIFIANT>/edit`.

2. Le compte de service — sur `https://console.cloud.google.com` avec le même compte :

1. Créer un projet (en haut à gauche → *Nouveau projet*, par exemple « rk-site »).
2. *API et services* → *Bibliothèque* → rechercher **Google Sheets API** → *Activer*.
3. *API et services* → *Identifiants* → *Créer des identifiants* → *Compte de service* : nom « rk-site-sync », aucun rôle à ajouter → OK.
4. Ouvrir le compte de service → onglet *Clés* → *Ajouter une clé* → *Créer une clé* → *JSON*. Un fichier `.json` est téléchargé : c'est un **secret**, ne jamais l'envoyer par e-mail, ni le publier, ni le déposer dans le dépôt Git.

3. Le partage — dans le tableur : *Partager* → coller l'adresse du compte de service (`rk-site-sync@<projet>.iam.gserviceaccount.com`, champ `client_email` du fichier JSON) → rôle **Éditeur** → décocher « Envoyer une notification » → *Partager*.

4. Les variables (`.env` en local, `.env.production` sur le serveur) :

```
GOOGLE_SHEETS_SPREADSHEET_ID=<identifiant du tableur>
GOOGLE_SERVICE_ACCOUNT_EMAIL=<client_email du fichier JSON>
GOOGLE_SERVICE_ACCOUNT_PRIVATE_KEY="<private_key du fichier JSON, sur une ligne, avec les \n>"
```

Recopier `private_key` telle qu'elle apparaît dans le fichier JSON (elle commence par `-----BEGIN PRIVATE KEY-----\n`), entre guillemets. Redémarrer le site, puis supprimer le fichier JSON téléchargé.

5. Vérifier — dans l'admin, *Demandes de contact* : le bloc **Google Sheets** apparaît avec **Ouvrir le tableur** et **Tout resynchroniser**. Cliquer sur *Tout resynchroniser* : les onglets « Demandes de contact » et « Abonnés newsletter » sont créés et remplis. En cas d'échec, le message rappelle l'adresse avec laquelle partager le tableur ; le détail (code Google, jamais les données) est dans les logs du serveur (`[sheets]`).

Fonctionnement : chaque création, modification ou suppression est recopiée en arrière-plan (une panne de Google ne bloque jamais le formulaire) ; les lignes sont retrouvées par la colonne **ID** ; les valeurs sont écrites en texte brut (aucune formule exécutée) ; les inscriptions non confirmées n'apparaissent pas ; les suppressions automatiques (durées de conservation) retirent aussi les lignes. **Changer ou révoquer la clé** : *Compte de service* → *Clés* → supprimer l'ancienne, en créer une nouvelle, mettre à jour la variable.

13. Checklist de mise en ligne

- ☐ Domaine pointé, HTTPS actif, redirection `http` → `https`
- ☐ `NEXT_PUBLIC_SITE_URL` correct et build effectué **après** son réglage
- ☐ `PAYLOAD_SECRET` unique, `CONTACT_RATE_LIMIT=5`
- ☐ SMTP configuré, demande de test envoyée et **reçue**
- ☐ Vente directe (si ouverte) : CGV et retours validés, TVA validée par le comptable, clés live Stripe/PayPal, webhook Stripe actif, achat test réel remboursé ; produits numériques : volume `private` sauvegardé, achat test → e-mail d'accès reçu, connexion et téléchargement vérifiés
- ☐ SPF, DKIM et DMARC publiés ; inscription newsletter testée de bout en bout (confirmation, article, désinscription)
- ☐ Google Sheets (si utilisé) : tableur partagé uniquement avec le compte de service, « Tout resynchroniser » réussi, fichier JSON de la clé supprimé du poste, politique de confidentialité mentionnant Google
- ☐ Google Analytics (si utilisé) : politiques de cookies et de confidentialité à jour ; en navigation privée, aucune requête Google avant « Accepter », puis visite visible dans *Temps réel* ; « Refuser » via « Paramètres des cookies » efface les cookies `_ga`
- ☐ Mot de passe administrateur changé, comptes inutiles supprimés
- ☐ Contenus « Contenu d'exemple » remplacés ou dépubliés
- ☐ Textes juridiques validés, case « brouillon » décochée
- ☐ `sitemap.xml` et `robots.txt` accessibles ; staging en `noindex`
- ☐ Sauvegarde automatique en place **et restauration testée**
- ☐ Sonde de disponibilité sur `/api/health` avec alerte e-mail
- ☐ Durée de conservation des demandes (`CONTACT_RETENTION_MONTHS`) identique à la politique de confidentialité
- ☐ Google Search Console : propriété vérifiée, sitemap soumis
- ☐ Accès et documentation remis au commanditaire